

09. Eval: String ko Code ki tarah chalana

Kya karta hai?

```
js
let result = eval(code);
eval('alert("Hello")');
eval('1+1'); // 2
```

- String ke andar ka code chala deta hai.
- Result = **aakhri statement ka result**.

Scope ka behavior

- Ye **current scope** me chalta hai, to bahar ke (outer) variables dekh bhi sakta hai aur badal bhi sakta hai:

```
js
let x = 5;
eval("x = 10");
console.log(x); // 10
```

- **Strict mode** me eval ka apna alag scope hota hai, andar declare kiye variables/functions bahar nahi dikhte.

"eval is evil"

Aajkal lagbhag zarurat nahi, kyunki:

- Code minifiers eval ki wajah se variable names chhote nahi kar paate, compression kharab hota hai.
- Outer local variables use karna bad practice hai, maintain karna mushkil.
- User ka input eval karna security risk hai.

Safe alternatives

1. Outer variables ki zarurat nahi to **global scope** me chalao:

```
js
window.eval('alert(x)');
```

2. Local data chahiye to `new Function` use karo aur arguments me pass karo:

js

```
let f = new Function('a', 'alert(a)');  
f(5);
```

Summary

- `eval(code)` string ko chalata hai.
- Modern JS me bahut kam use hota hai.
- Global scope ke liye `window.eval`, data ke saath ke liye `new Function`.